

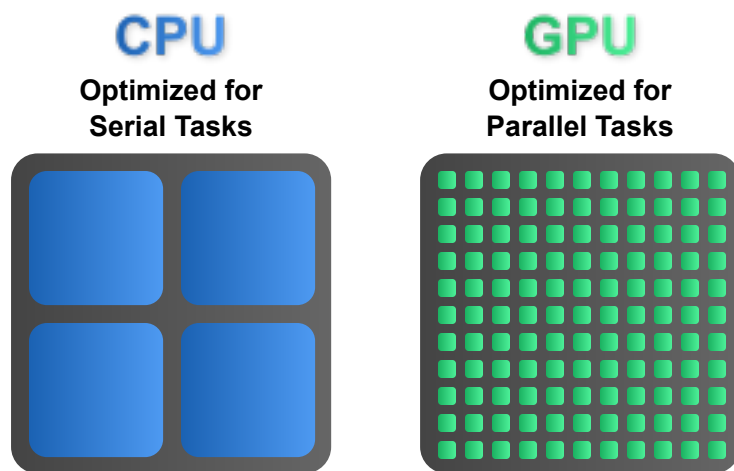
MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Teknologi Kecerdasan Artifisial (Tingkat 1)

Pertemuan I

Kemampuan CPU vs GPU

Pemrosesan data antara penggunaan CPU dan GPU sangat berbeda. Di bawah ini adalah contoh program dengan menggunakan python 3 untuk melihat perbandingan kemampuan antara penggunaan CPU dengan GPU.



Program:

Di bawah ini adalah contoh program python menggunakan library numpy yang memanfaatkan CPU dan juga cupy yang memanfaatkan GPU. Sementara library time digunakan untuk melakukan perhitungan waktu yang diperlukan.

```
In [1]: import numpy as np
import cupy as cp
import time
```

Kode di bawah ini membuat array 3D dengan jumlah 1 Miliar, untuk Numpy dan CuPy.

```
In [2]: ### Numpy dan CPU  
s = time.time()  
x_cpu = np.ones((1000,1000,1000))  
e = time.time()  
print("Waktu yang diperlukan untuk CPU:", e - s)
```

Waktu yang diperlukan untuk CPU: 0.8750002384185791

```
In [3]: ### CuPy dan GPU
s = time.time()
x_gpu = cp.ones((1000,1000,1000))
cp.cuda.Stream.null.synchronize()
e = time.time()
print("Waktu yang diperlukan untuk GPU:", e - s)
```

```

-----
-
OutOfMemoryError                                Traceback (most recent call las
t)
Cell In[3], line 3
      1 ### CuPy dan GPU
      2 s = time.time()
----> 3 x_gpu = cp.ones((1000,1000,1000))
      4 cp.cuda.Stream.null.synchronize()
      5 e = time.time()

File /usr/local/lib/python3.10/site-packages/cupy/_creation/basic.py:183,
in ones(shape, dtype, order)
    162 def ones(
    163     shape: _ShapeLike,
    164     dtype: DTypeLike = float,
    165     order: _OrderCF = 'C',
    166 ) -> NDArray[Any]:
    167     """Returns a new array of given shape and dtype, filled with o
nes.
    168
    169     This function currently does not support ``order`` option.
    (...)
    181
    182     """
--> 183     a = cupy.ndarray(shape, dtype, order=order)
    184     a.fill(1)
    185     return a

File cupy/_core/core.pyx:167, in cupy._core.core.ndarray.__new__()

File cupy/_core/core.pyx:254, in cupy._core.core._ndarray_base._init()

File cupy/cuda/memory.pyx:875, in cupy.cuda.memory.alloc()

File cupy/cuda/memory.pyx:1579, in cupy.cuda.memory.MemoryPool.malloc()

File cupy/cuda/memory.pyx:1600, in cupy.cuda.memory.MemoryPool.malloc()

File cupy/cuda/memory.pyx:1271, in cupy.cuda.memory.SingleDeviceMemoryPoo
l.malloc()

File cupy/cuda/memory.pyx:1292, in cupy.cuda.memory.SingleDeviceMemoryPoo
l._malloc()

File cupy/cuda/memory.pyx:1537, in cupy.cuda.memory.SingleDeviceMemoryPoo
l._try_malloc()

File cupy/cuda/memory.pyx:1540, in cupy.cuda.memory.SingleDeviceMemoryPoo
l._try_malloc()

OutOfMemoryError: Out of memory allocating 8,000,000,000 bytes (allocated
so far: 0 bytes).

```

Dari hasil di atas dapat dilihat perbedaan hasil kecepatan waktu antara penggunaan CPU dan GPU. Program berikutnya untuk melakukan operasi matematika terhadap array yaitu dengan mengkalikannya dengan nilai tertentu.

```
In [4]: ### Nilai yang akan dikalikan dengan array  
n = 5
```

```
In [5]: ### Numpy dan CPU  
s = time.time()  
x_cpu *= n  
e = time.time()  
print("Waktu yang diperlukan untuk CPU:", e - s)
```

Waktu yang diperlukan untuk CPU: 1.6238508224487305

```
In [6]: ### CuPy and GPU  
s = time.time()  
x_gpu *= n  
cp.cuda.Stream.null.synchronize()  
e = time.time()  
print("Waktu yang diperlukan untuk GPU:", e - s)
```

```
-----  
-  
NameError                                Traceback (most recent call las  
t)  
Cell In[6], line 3  
      1 ### CuPy and GPU  
      2 s = time.time()  
----> 3 x_gpu *= n  
      4 cp.cuda.Stream.null.synchronize()  
      5 e = time.time()  
  
NameError: name 'x_gpu' is not defined
```

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma
<https://www.praktikum-hpc.gunadarma.ac.id/> (<https://www.praktikum-hpc.gunadarma.ac.id/>)